

Punteros

En informática, un puntero es una variable especializada que, en lugar de almacenar un dato común (como un número o una letra), contiene la dirección de memoria de otra variable. Es como tener una nota que no dice el valor de algo, sino que te indica exactamente en qué "casillero" de la memoria RAM se encuentra guardado ese valor.

Declaración

Para declarar un puntero en lenguajes como C o C++, se utiliza el símbolo asterisco (*) junto al tipo de dato al que se desea apuntar. Esto le indica al compilador que la variable no guardará un valor de ese tipo, sino la ubicación de uno.

ejemplo: `int *p;` declara un puntero llamado p que puede apuntar a cualquier variable de tipo entero.

Operadores principales

Existen dos operadores fundamentales para trabajar con punteros:

Operador de dirección (&): Se coloca antes de una variable normal para obtener su dirección de memoria (su "ubicación").

Operador de indirección o desreferencia (*): Se usa con un puntero para acceder al valor real que reside en la dirección que el puntero tiene guardada.

Operaciones y aritmética

Los punteros permiten realizar operaciones específicas que facilitan el manejo de datos complejos:

Asignación: Puedes hacer que un puntero apunte a una variable usando el operador &. Por ejemplo: `p = №`

Aritmética de punteros: Se pueden sumar o restar valores enteros a un puntero. Esto es común al trabajar con arreglos (vectors), ya que sumar 1 al puntero lo desplaza a la siguiente posición de memoria del mismo tipo.

Paso por referencia: Permite que una función modifique directamente el valor de una variable externa pasando su dirección (puntero) en lugar de una copia del valor.

Punteros y Funciones

Cuando pasas un puntero a una función, realizas un paso por referencia. A diferencia del paso por valor (donde la función recibe una copia), aquí le entregas la dirección real de la variable. Esto permite que cualquier cambio hecho dentro de la función afecte permanentemente a la variable original fuera de ella, ahorrando memoria al no duplicar datos grandes.

Ejemplo: Una función que intercambia dos números necesita punteros para modificar las variables originales.

```
void duplicar(int *n) {  
    *n = (*n) * 2; (Accede al valor real y lo multiplica)
```

```
}
```

(Uso: duplicar(&miNumero);)

Punteros y Estructuras

Los punteros son fundamentales para manejar estructuras (structs), especialmente cuando son complejas. Para acceder a los miembros de una estructura a través de un puntero, se utiliza el operador de flecha (->). Este operador es una forma abreviada de desreferenciar el puntero y acceder al campo al mismo tiempo, lo que hace el código mucho más limpio y legible.

Ejemplo: Si tienes una estructura Persona, puedes modificar sus datos mediante un puntero.

```
struct Persona {  
    char nombre[20];  
    int edad;  
};
```

```
struct Persona p1 = {"Ana", 25};  
struct Persona *ptr = &p1;
```

ptr->edad = 26; (Cambia la edad de Ana a 26 usando el puntero)

Listas Enlazadas

Las listas enlazadas son estructuras de datos dinámicas compuestas por una serie de nodos. A diferencia de los arreglos, los elementos de una lista no están almacenados en posiciones contiguas de memoria; en su lugar, cada nodo contiene el dato y un puntero que indica la dirección del siguiente elemento. Se clasifican principalmente en simplemente enlazadas, doblemente enlazadas y circulares (simples o dobles).

Listas Simplemente Enlazadas

Se basan en un flujo unidireccional. Cada nodo tiene un solo enlace hacia el sucesor. El último nodo de la lista apunta a NULL, indicando el final de la estructura. Su mayor ventaja es la flexibilidad para crecer o reducirse en tiempo de ejecución sin desperdiciar memoria.

Punteros de Cabecera y Cola: La Cabecera (head) es el puntero más crítico, ya que guarda la dirección del primer nodo; si se pierde, se pierde el acceso a toda la lista. La Cola (tail) es opcional y apunta al último nodo para permitir inserciones rápidas al final.

Operador de Selección: Se utiliza el operador flecha (->) para acceder a los campos dato y siguiente del nodo a través de su puntero.

Operaciones Básicas:

Declaración: Se define un struct que contiene el dato y un puntero al mismo tipo de struct.

Inserción: Para insertar al inicio, el nuevo nodo apunta a la actual cabecera y luego la cabecera se actualiza al nuevo nodo.

Búsqueda: Se recorre la lista con un puntero auxiliar desde la cabecera hasta encontrar el valor o llegar a NULL.

Eliminación: Se localiza el nodo, se hace que el nodo anterior apunte al siguiente del que se va a borrar, y se libera la memoria del nodo eliminado.

Ejemplo (Inserción al inicio):

```
struct Nodo {
    int dato;
    struct Nodo *siguiente;
};

void insertarInicio(struct Nodo **cabecera, int valor) {
    struct Nodo *nuevo = malloc(sizeof(struct Nodo));
    nuevo->dato = valor;
    nuevo->siguiente = *cabecera;
    *cabecera = nuevo;
}
```

Listas Doblemente Enlazadas

En estas listas, cada nodo posee dos punteros: uno al nodo siguiente (sig) y otro al anterior (ant). Esto permite recorrer la lista en ambas direcciones y facilita la eliminación de nodos, ya que no es necesario buscar al "padre" manualmente.

Declaración y Recorrido: El struct incluye *sig y *ant. El recorrido puede ser hacia adelante hasta NULL o hacia atrás desde la cola.

Inserción y Eliminación: Son más complejas porque requieren actualizar cuatro punteros (dos del nuevo nodo y uno de cada vecino) para mantener la integridad de la cadena.

Ejemplo (Estructura doble):

```
struct NodoDoble {
    int dato;
    struct NodoDoble *sig, *ant;
};
```

Listas Circulares

Son variantes donde no existe un final nulo. En una circular simple, el último nodo apunta de nuevo a la cabecera. En una circular doble, además, la cabecera apunta hacia atrás al último nodo.

Recorrido: El ciclo de parada no es NULL, sino cuando el puntero auxiliar vuelve a ser igual a la cabecera.

Inserción/Eliminación: Es vital actualizar el enlace del último nodo cada vez que la cabecera cambia para no romper el anillo.

Recursividad

La recursividad es una técnica de programación donde una función se llama a sí misma para resolver un problema. El fundamento teórico se basa en la estrategia de "divide y vencerás": un problema complejo se descompone en versiones más pequeñas e idénticas de sí mismo hasta llegar a un caso base, que es una situación lo

suficientemente simple como para resolverse directamente sin más llamadas.

Su ámbito de aplicación es vasto, destacando en el procesamiento de estructuras de datos jerárquicas (como árboles y grafos), algoritmos de ordenamiento (QuickSort, MergeSort) y problemas matemáticos definidos de forma inductiva. Su utilidad principal radica en que permite escribir soluciones elegantes y compactas para problemas que, de forma iterativa (con bucles), serían mucho más difíciles de leer y mantener.

Ventajas y Desventajas

Como toda herramienta, la recursividad tiene un costo y un beneficio:

Ventajas: El código suele ser más corto, limpio y cercano a la notación matemática. Facilita la resolución de problemas que tienen una naturaleza recursiva inherente, como recorrer todas las carpetas y subcarpetas de un disco duro.

Desventajas: Cada llamada recursiva consume espacio en la pila de ejecución (stack), lo que puede provocar un error de "Stack Overflow" si la profundidad es excesiva. Además, suele ser menos eficiente en tiempo de ejecución que un bucle simple debido a la sobrecarga de crear nuevos contextos de función.

Diseño y Escritura de Programas Recursivos

Para diseñar un programa recursivo correctamente, se deben definir dos partes críticas:

Caso Base: La condición de parada que detiene las llamadas y comienza a devolver los resultados. Sin esto, el programa entra en un bucle infinito.

Caso Recursivo: La parte donde la función se llama a sí misma, pero siempre con un argumento que la acerque un paso más al caso base.

Ejemplo clásico: El Factorial

El factorial de n ($n!$) es $n \times (n-1)!$. El caso base es $0! = 1$.

```
int factorial(int n) {  
    (1. Caso Base)  
    if (n == 0) {  
        return 1;  
    }  
  
    (2. Caso Recursivo)  
    else {  
        return n * factorial(n - 1);  
    }  
}
```

Ejemplo de Recorrido: Imprimir una Lista Enlazada

La recursividad es ideal para estructuras de datos:

```
void imprimirLista(struct Nodo *n) {  
    if (n == NULL) return; (Caso base: lista vacía)  
    printf("%d ", n->dato);  
}
```

```
    imprimirLista(n->siguiente); (Caso recursivo: el resto de la lista)
}
```

Introducción a las Estructuras de Datos Dinámicas Avanzadas: Pilas, Colas y Árboles

Las estructuras de datos dinámicas avanzadas permiten gestionar colecciones de datos cuya memoria se asigna en tiempo de ejecución, adaptándose al volumen de información necesario. A diferencia de las listas lineales simples, estructuras como las pilas, colas y árboles imponen reglas específicas de acceso y organización para resolver problemas de lógica de datos más complejos.

Pilas (Stacks)

Una pila es una estructura de tipo LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir. Imagina una pila de platos: solo puedes añadir o quitar el que está arriba. Su especificación principal es que todas las inserciones y extracciones ocurren en un único extremo llamado "Tope".

Funcionalidades: push (insertar), pop (extraer) y peek (ver el tope).

Implementación: Se usa un puntero al nodo superior. Al insertar, el nuevo nodo apunta al antiguo tope y el puntero "Tope" se actualiza al nuevo.

Ejemplo: El botón "Atrás" del navegador o la función "Deshacer" en un editor de texto.

Colas (Queues)

La cola sigue el principio FIFO (First In, First Out), donde el primero en llegar es el primero en ser atendido, como la fila de un banco. Su definición implica dos extremos: el "Frente" para extraer y el "Final" para insertar. Existen tipos como las colas de prioridad (elementos con urgencia) y las colas circulares.

Funcionalidades: enqueue (encolar) y dequeue (desencolar).

Implementación: Requiere dos punteros. Al encolar, se añade al final; al desencolar, se mueve el puntero del frente al siguiente nodo.

Ejemplo: Una cola de impresión donde los documentos se imprimen en el orden en que se enviaron.

Árboles (Trees)

Un árbol es una estructura no lineal y jerárquica. El tipo más común es el Árbol Binario, donde cada nodo tiene como máximo dos hijos (izquierdo y derecho). Se clasifican en árboles de búsqueda (BST), balanceados (AVL) o n-arios. La especificación clave es el nodo "Raíz", desde donde nace toda la estructura.

Funcionalidades: Inserción jerárquica y recorridos (Inorden, Preorden, Postorden).

Implementación: Cada nodo es un struct con el dato y dos punteros: *izq y *der.

Ejemplo: El sistema de archivos de una computadora (Carpetas > Subcarpetas > Archivos).

Ejemplo de implementación básica (Nodo de Árbol Binario):

```
struct NodoArbol {  
    int info;  
    struct NodoArbol *izq;  
    struct NodoArbol *der;  
};
```

Para insertar, se decide si el dato va a la izquierda (si es menor) // o a la derecha (si es mayor) de forma recursiva.